

M A S T E R T H E S I S

Rohfassung & Strukturentwurf

# The Asymmetry of Verification in Enterprise SaaS Sales

*Applying Jason Wei's Verification Rule to Real-World Use Cases  
and Deriving Design Patterns for Human-AI Interfaces*

---

Johannes Hötter

Februar 2026

# Inhaltsverzeichnis

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Inhaltsverzeichnis .....                                                 | 2  |
| 1 Einleitung & Motivation .....                                          | 4  |
| 1.1 Warum Enterprise SaaS Sales? .....                                   | 4  |
| 1.2 Zwei Dimensionen der Forschungsfrage .....                           | 5  |
| 2 Theoretische Grundlagen & Related Work .....                           | 7  |
| 2.1 The Asymmetry of Verification - Jason Wei .....                      | 7  |
| 2.2 Design Science Research (DSR) .....                                  | 8  |
| 2.3 DSR mit Echelons (Tuunanen et al.) .....                             | 8  |
| 2.4 Enterprise SaaS Sales - Domäne und Kontext .....                     | 9  |
| 2.5 Weitere relevante Forschungsfelder .....                             | 10 |
| 3 Forschungsfrage & Zielsetzung .....                                    | 11 |
| 3.1 Die How-Might-We-Question .....                                      | 11 |
| 3.2 Erwartete Ergebnisse .....                                           | 11 |
| 4 Methodisches Vorgehen .....                                            | 12 |
| 4.1 Überblick: DSR mit Echelons .....                                    | 12 |
| 4.2 Forschungsprozess im Detail .....                                    | 12 |
| Phase 1: Exploration & Problemverständnis .....                          | 12 |
| Phase 2: Synthese & Strukturierung .....                                 | 12 |
| Phase 3: Hypothesenbildung & Prototyping .....                           | 13 |
| Phase 4: Evaluation & Iteration .....                                    | 13 |
| 4.3 Zwei-Ebenen-Iteration .....                                          | 13 |
| 5 Der Verifikationsquadrant im Enterprise SaaS Sales .....               | 14 |
| 5.1 Ausgangslage: Der leere Quadrant .....                               | 14 |
| 5.2 Erste Exploration: Interviews mit Enterprise-Vertriebspersonen ..... | 14 |
| Zentrale Erkenntnis: Drei Töpfe .....                                    | 14 |
| 5.3 Einordnung in den Quadranten .....                                   | 15 |
| Quadrant B: Schwer zu generieren, leicht zu verifizieren .....           | 15 |
| Quadrant C: Leicht zu generieren, schwer zu verifizieren .....           | 16 |
| Quadrant D: Schwer zu generieren, schwer zu verifizieren .....           | 16 |
| Quadrant A: Leicht zu generieren, leicht zu verifizieren .....           | 16 |
| 5.4 Iterationen auf der Use-Case-Ebene .....                             | 16 |
| 5.4.1 Security Questionnaires - Echelons .....                           | 17 |
| 5.4.2 NDA Review & Approval - Echelons .....                             | 17 |
| 5.4.3 ROI-Case-Erstellung - Echelons .....                               | 18 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 5.5 Iteration auf der Quadranten-Ebene .....                    | 18 |
| 6 Design Patterns für Human-AI Interfaces .....                 | 20 |
| 6.1 Pattern 1: Notification Inbox (Quadrant B) .....            | 20 |
| 6.2 Pattern 2: Guided Analysis Workspace (Quadrant C) .....     | 21 |
| 6.3 Pattern 3: Auto-Execute with Audit Trail (Quadrant A) ..... | 21 |
| 6.4 Pattern 4: Structured Co-Creation (Quadrant D) .....        | 22 |
| 6.5 Zusammenfassung: Quadrant → Pattern Mapping .....           | 22 |
| 7 Diskussion & weiterführende Implikationen .....               | 23 |
| 7.1 Verschiebung des Tätigkeitsprofils .....                    | 23 |
| 7.2 Human-in-the-Loop vs. AI-in-the-Loop .....                  | 23 |
| 7.3 Agentic Business Process Management .....                   | 23 |
| 7.4 Guardrails & Vertrauen .....                                | 24 |
| 7.5 Wertschöpfungsketten und Kaskadeneffekte .....              | 24 |
| 8 Fazit & Ausblick .....                                        | 25 |
| Anhang: Strategie für die Verteidigung .....                    | 26 |
| Vorgeschlagener Aufbau der Präsentation (ca. 20-25 Min.) .....  | 26 |

# 1 Einleitung & Motivation

Die Leistungsfähigkeit moderner KI-Systeme, insbesondere Large Language Models (LLMs), hat in den vergangenen Jahren eine bemerkenswerte Dynamik entwickelt. Was einst als Zukunftsvision galt – die automatisierte Erstellung von Quellcode, die Beantwortung komplexer Fachfragen, die Analyse unstrukturierter Daten – ist heute bereits Realität. Doch während die Fähigkeit zur **Generierung** von Lösungen rasant voranschreitet, bleibt eine fundamentale Frage oft unterbelichtet: Wie einfach oder schwierig ist es eigentlich, eine generierte Lösung zu **verifizieren**?

Diese Frage ist keineswegs trivial. Jason Wei, ein renommierter KI-Forscher, hat in seinem viel beachteten Blogpost diese Asymmetrie zwischen Generierung und Verifikation systematisch aufgearbeitet. Seine zentrale Beobachtung lautet: Nicht alle Aufgaben sind gleich. Bei manchen ist es schwer, eine Lösung zu finden, aber einfach zu prüfen, ob sie korrekt ist. Bei anderen verhält es sich genau umgekehrt – oder es ist beides schwierig.

Das vielleicht anschaulichste Beispiel ist das **Sudoku**: Eine Lösung zu generieren erfordert systematisches Durchprobieren und logisches Denken – das ist schwer. Aber sobald eine Lösung vorliegt, kann jeder Mensch (oder jedes Programm) in Sekunden prüfen, ob alle Zeilen, Spalten und Blöcke korrekt gefüllt sind. Die Verifikation ist trivial. Am anderen Ende des Spektrums steht die **Riemannsche Vermutung**: Hier ist sowohl die Generierung eines Beweises als auch dessen Verifikation extrem schwierig – seit über 150 Jahren unbewiesen.

Besonders spannend wird es bei Aufgaben, die **einfach zu behaupten, aber schwer zu verifizieren** sind. Wenn jemand sagt: „Wenn du nur noch Karotten und Kartoffeln isst, wirst du 100 Jahre alt“, dann ist die Behauptung in Sekunden aufgestellt – die Verifikation dauert hingegen Jahrzehnte und erfordert umfangreiche empirische Daten. Genau diese Asymmetrie ist für den Einsatz von KI in der Praxis hochrelevant.

## 1.1 Warum Enterprise SaaS Sales?

Diese Arbeit nimmt einen sehr konkreten, praxisnahen Anwendungsfall unter die Lupe: den Enterprise Software-as-a-Service (SaaS) Vertrieb. Dieser Bereich wurde gewählt, weil er eine außergewöhnliche Bandbreite an Aufgabentypen vereint, die

sich fundamental in ihrer Generierungs- und Verifikationsschwierigkeit unterscheiden.

Enterprise SaaS Sales – also der Verkauf hochpreisiger Software-Lösungen an Großunternehmen mit **sechsstelligen Average Contract Values** – ist ein Bereich, der sich grundlegend von Low-Value-Sales unterscheidet. Hier reicht keine Kreditkarte. Stattdessen sind die Prozesse geprägt von:

- **Hoher Komplexität:** Mehrere Stakeholder, lange Entscheidungszyklen, komplexe Organisationsstrukturen
- **Intensiver Kommunikation:** Zwischen Vertrieb und Kunde, aber auch intern zwischen Vertrieb und Fachexperten (Security, Legal, Product)
- **Vielfältigen Aufgabentypen:** Von vertraglichen Themen über Mehrwert-Argumentation bis hin zu Customer-Success-Aufgaben
- **Hohem Druck:** Jeder Deal ist strategisch relevant, Fehler sind kostspielig

Die zentrale Frage dieser Arbeit lautet daher:

***How might we apply the Verification Rule to a real-world application such as Enterprise SaaS Sales – and derive design patterns for human-AI interfaces based on where use cases fall in the verification quadrant?***

## 1.2 Zwei Dimensionen der Forschungsfrage

Die How-Might-We-Question dieser Arbeit hat zwei ineinander verwobene Dimensionen:

1. **Anwendung der Verification Rule:** Wie lassen sich die verschiedenen Aufgaben im Enterprise SaaS Vertrieb systematisch in den Quadranten der Verifikationsasymmetrie einordnen? Was bedeutet das für die Frage, welche Aufgaben KI kurzfristig übernehmen kann – und wo der Mensch weiterhin essenziell bleibt?
2. **Ableitung von Design Patterns:** Lassen sich aus der Position eines Use Cases im Quadranten systematische Designmuster für Software-Interfaces ableiten? Gibt es wiederkehrende Muster, wie Interfaces für „leicht zu verifizieren“ vs. „schwer zu verifizieren“ aussehen sollten?

Diese Arbeit verbindet damit theoretische Grundlagen aus der KI-Forschung mit praktischer Designarbeit und liefert sowohl wissenschaftliche Erkenntnisse als auch unmittelbar anwendbare Gestaltungsprinzipien.

## 2 Theoretische Grundlagen & Related Work

### 2.1 The Asymmetry of Verification – Jason Wei

Den konzeptionellen Ausgangspunkt dieser Arbeit bildet Jason Weis Arbeit zur **Asymmetrie der Verifikation**. Wei argumentiert, dass die Schwierigkeit einer Aufgabe nicht eindimensional betrachtet werden kann. Stattdessen müssen zwei orthogonale Dimensionen unterschieden werden:

1. **Generierungsschwierigkeit (Generation Difficulty):** Wie schwer ist es, überhaupt eine Lösung oder Antwort zu produzieren?
2. **Verifikationsschwierigkeit (Verification Difficulty):** Wie schwer ist es, eine vorliegende Lösung auf Korrektheit zu prüfen?

Aus dieser Zweiteilung ergibt sich ein Quadrant mit vier grundlegend verschiedenen Aufgabentypen:

| Schwer zu generieren Leicht zu verifizieren                                                                                                                                                                                       | Schwer zu generieren Schwer zu verifizieren                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Quadrant B</b> <ul style="list-style-type: none"> <li>• Sudoku-Typ-Aufgaben</li> <li>• Security Questionnaires</li> <li>• NDA Review / Approval</li> <li>• Contract Redlining</li> <li>• Support-Ticket-Bearbeitung</li> </ul> | <b>Quadrant D</b> <ul style="list-style-type: none"> <li>• Mathematische Beweise</li> <li>• ROI-Case-Erstellung</li> <li>• Langzeit-Kundenwertsteigerung</li> </ul>        |
| <b>Quadrant A</b> <ul style="list-style-type: none"> <li>• Einfache Statusabfragen</li> <li>• Standard-E-Mail-Vorlagen</li> <li>• Meeting-Zusammenfassungen</li> </ul>                                                            | <b>Quadrant C</b> <ul style="list-style-type: none"> <li>• Claim: Unser Produkt spart 30%</li> <li>• Prognosen &amp; Forecasts</li> <li>• Org.-Struktur-Analyse</li> </ul> |
| Leicht zu generieren Leicht zu verifizieren                                                                                                                                                                                       | Leicht zu generieren Schwer zu verifizieren                                                                                                                                |

Weis zentrale These – die „**Verifiers Rule**“ – besagt: KI wird besonders dort schnell Fortschritte machen, wo die Verifikation einfach ist. Der Grund ist fundamental: Wenn wir schnell prüfen können, ob eine generierte Lösung korrekt ist, können wir die KI effizient trainieren, iterieren und einsetzen. **Die Qualität der Verifikation bestimmt die Qualität der Generierung.**

Dieses Prinzip lässt sich auch auf menschliche Arbeitsprozesse übertragen: Aufgaben mit einfacher Verifikation eignen sich hervorragend für eine Mensch-

Maschine-Arbeitsteilung, bei der die KI generiert und der Mensch verifiziert. Bei Aufgaben mit schwieriger Verifikation hingegen stellt sich die Frage, ob und wie ein menschlicher Reviewer überhaupt sinnvoll eingebunden werden kann.

**Implikation für die Softwareentwicklung:** Wei weist auch darauf hin, dass das Verhältnis veränderbar ist. In der Softwareentwicklung beispielsweise können Unit-Tests die Verifikation automatisieren – damit verschiebt sich eine Aufgabe im Quadranten. Dieser Gedanke ist für die vorliegende Arbeit zentral: Durch geschicktes Interface-Design und Automatisierungsgestaltung können wir die Verifikation erleichtern und damit die Einsatzfähigkeit von KI verbessern.

## 2.2 Design Science Research (DSR)

Als methodisches Fundament dient **Design Science Research (DSR)** nach Hevner et al. DSR ist ein Forschungsparadigma, das sich von klassischer empirischer Forschung dadurch unterscheidet, dass es nicht nur Phänomene beschreibt und erklärt, sondern aktiv **Artefakte** entwickelt – Software-Prototypen, Frameworks, Design Principles – und deren Wirksamkeit evaluiert.

DSR eignet sich ideal für diese Arbeit, weil:

- Es die iterative Entwicklung von Software-Artefakten wissenschaftlich rahmt
- Es die Verbindung von Theorie (Verification Rule) und Praxis (Interface-Design) ermöglicht
- Es explizit Evaluation als integralen Bestandteil vorsieht
- Es den praxisnahen Charakter der Forschungsfrage abbildet

## 2.3 DSR mit Echelons (Tuunanen et al.)

Für die konkrete Durchführung wird eine Erweiterung von DSR verwendet: **Design Science Research mit Echelons** nach Tuunanen et al. Ein Echelon ist dabei eine abgegrenzte, überschaubare Iteration von Forschung und Entwicklung, bestehend aus:

1. **Exploration:** Interviews, Literaturrecherche, Shadowing, eigene Erfahrungen
2. **Analyse & Strukturierung:** Mindmaps, Requirements Engineering, Synthese der Erkenntnisse



3. **Hypothesenbildung:** Erstellung von Mockups, Prototypen oder Software-Artefakten
4. **Evaluation:** Testing, Interviews, Feedback, Einsatz in der Praxis
5. **Lernen & Iteration:** Erkenntnisse extrahieren, Hypothesen anpassen, nächste Iteration starten

Der entscheidende Vorteil gegenüber einem linearen Vorgehen: Echelons können auf **verschiedenen Abstraktionsebenen** stattfinden. In dieser Arbeit wird auf zwei Ebenen iteriert:

- **Use-Case-Ebene:** Einzelne Use Cases (z.B. Security Questionnaires) werden in eigenen Echelons erforscht, prototypisch umgesetzt und evaluiert.
- **Quadranten-Ebene:** Auf einer höheren Abstraktionsebene werden Gemeinsamkeiten zwischen Use Cases desselben Quadranten identifiziert und zu übergreifenden Design Patterns verdichtet.

Dieses Vorgehen ermöglicht es, sowohl in die Tiefe einzelner Use Cases zu gehen als auch das große Bild – den gefüllten Quadranten mit abgeleiteten Design Patterns – schrittweise zu entwickeln.

## 2.4 Enterprise SaaS Sales – Domäne und Kontext

Enterprise SaaS Sales bezeichnet den Vertrieb von Software-as-a-Service-Lösungen an Großunternehmen (Enterprises). Im Gegensatz zu Self-Service- oder SMB-Vertrieb zeichnet sich dieser Bereich durch **hohe Komplexität, lange Verkaufszyklen und viele Stakeholder** aus.

Relevante Literatur und Frameworks für diese Domäne:

- **MEDDIC/MEDDICC:** Framework zur Qualifizierung von Enterprise-Deals (Metrics, Economic Buyer, Decision Criteria, Decision Process, Identify Pain, Champion, Competition)
- **SPIN Selling (Rackham):** Fragetechnik für komplexe Verkäufe (Situation, Problem, Implication, Need-Payoff)
- **Challenger Sale (Dixon & Adamson):** Ansatz, der den Vertriebler als Berater positioniert, der dem Kunden neue Perspektiven eröffnet

- **Revenue Operations:** Moderne Ansätze zur Integration von Sales, Marketing und Customer Success

## 2.5 Weitere relevante Forschungsfelder

Die Arbeit berührt zudem mehrere angrenzende Forschungsfelder, die in der Diskussion aufgegriffen werden:

- **Agentic Business Process Management (BPM):** Wie können KI-Agenten in kommunikationsintensive Geschäftsprozesse eingebunden werden?
- **Human-AI Collaboration:** Wie gestaltet man die Zusammenarbeit zwischen Mensch und KI optimal? Wann ist ein Human-in-the-Loop sinnvoll – und wann wird er zum Bottleneck?
- **Guardrails & AI Safety:** Wie können Leitplanken gestaltet werden, die Qualität sichern, ohne den Prozess unnötig zu verlangsamen?
- **Wertschöpfungsketten & Task Shifting:** Wenn KI bestimmte Aufgaben übernimmt, wie verschiebt sich das Tätigkeitsprofil des Menschen?

## 3 Forschungsfrage & Zielsetzung

### 3.1 Die How-Might-We-Question

Die zentrale Forschungsfrage dieser Arbeit wird als How-Might-We-Question formuliert, was den explorativen und gestaltenden Charakter der Arbeit unterstreicht:

**How might we apply the Verification Rule to Enterprise SaaS Sales  
and derive design patterns for human-AI interfaces?**

### 3.2 Erwartete Ergebnisse

Die Arbeit zielt auf drei konkrete Ergebnisse ab:

3. **Gefüllter Verifikationsquadrant:** Eine systematische Einordnung der identifizierten Use Cases aus dem Enterprise SaaS Vertrieb in den Quadranten. Dieser Quadrant wird im Verlauf der Arbeit schrittweise aufgebaut – der Leser erlebt, wie sich das Bild durch die iterativen Forschungsschleifen vervollständigt.
4. **Design Patterns pro Quadrant:** Für jeden Quadranten (oder zumindest die am stärksten besetzten) werden Interface-Designmuster abgeleitet. Diese Patterns beschreiben, wie Software gestaltet sein sollte, um die jeweilige Kombination aus Generierungs- und Verifikationsschwierigkeit optimal zu unterstützen.
5. **Implikationen & Diskussion:** Einordnung der Ergebnisse in den breiteren Kontext von Human-AI Collaboration, Agentic BPM und die Zukunft der Vertriebsarbeit.

## 4 Methodisches Vorgehen

### 4.1 Überblick: DSR mit Echelons

Das methodische Vorgehen folgt dem Rahmenwerk von **Design Science Research mit Echelons** (Tuunanen et al.), aufbauend auf den Grundprinzipien von Hevner et al. Der Forschungsprozess gliedert sich in mehrere aufeinander aufbauende Iterationsschleifen (Echelons), die auf zwei Abstraktionsebenen stattfinden.

**Ausgangslage:** Enterprise SaaS Vertrieb ist ein komplexes, kommunikationsintensives Tätigkeitsfeld mit heterogenen Aufgabentypen. Die Vermutung besteht, dass Jason Weis Verification Rule einen wertvollen analytischen Rahmen bietet, um diese Aufgaben systematisch zu strukturieren und daraus Designimplikationen abzuleiten.

### 4.2 Forschungsprozess im Detail

#### Phase 1: Exploration & Problemverständnis

In der initialen Explorationsphase werden verschiedene Methoden kombiniert, um ein tiefes Verständnis der Domäne zu entwickeln:

- **Explorative Interviews (n=7+):** Halbstrukturierte Gespräche mit Vertriebspersonen, die Enterprise SaaS mit sechsstelligen ACVs verkaufen. Durchgehen konkreter Deal-Verläufe, Identifikation von Schmerzpunkten, Aufgabentypen und Zeitfressern.
- **Literaturrecherche:** Systematische Aufarbeitung von Sales-Methodik-Literatur (MEDDIC, SPIN, Challenger), KI-in-Sales-Forschung und Design Science Research.
- **Eigene Domänenenerfahrung:** Reflexion eigener Erfahrungen im Enterprise SaaS Umfeld als zusätzliche Datenquelle.
- **Shadowing (optional):** Begleitung von Vertriebspersonen in ihrem Arbeitsalltag zur Beobachtung tatsächlicher Workflows.

#### Phase 2: Synthese & Strukturierung

Die Erkenntnisse aus Phase 1 werden strukturiert und verdichtet:

- Erstellung einer Mindmap aller identifizierten Aufgabentypen
- Kategorisierung in die drei Hauptbereiche: Vertragliches, Mehrwert-Argumentation, Customer Success
- Erste Einordnung der Use Cases in den Verifikationsquadranten
- Ableitung von Requirements und Hypothesen für erste Prototypen

### Phase 3: Hypothesenbildung & Prototyping

Basierend auf den Erkenntnissen werden Software-Artefakte erstellt:

- Mockups und Wireframes für verschiedene Interface-Ansätze
- Funktionierende Prototypen für ausgewählte Use Cases
- Insgesamt vier Iterationsversionen, die zunehmend konvergieren

### Phase 4: Evaluation & Iteration

Jeder Prototyp wird evaluiert durch:

- Evaluierungsinterviews mit den ursprünglichen Interviewpartnern
- Praktischen Einsatz der Prototypen in realen Szenarien
- Feedback-Analyse und Ableitung von Learnings
- Rückbezug auf den Quadranten: Bestätigt sich die Einordnung?

## 4.3 Zwei-Ebenen-Iteration

Ein zentrales Merkmal des Vorgehens ist die Iteration auf zwei Ebenen:

| Use-Case-Ebene                                                                                                                                                                                                                                                                         | Quadranten-Ebene                                                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Einzelne Use Cases werden in eigenen Echelons untersucht:</p> <ul style="list-style-type: none"><li>• Security Questionnaire bearbeiten</li><li>• NDA prüfen und freigeben</li><li>• ROI-Case aufbauen</li><li>• Support-Tickets lösen</li><li>• Org-Struktur analysieren</li></ul> | <p>Gemeinsamkeiten zwischen Use Cases eines Quadranten:</p> <ul style="list-style-type: none"><li>• Was haben alle „leicht zu verifizieren“-UCs gemeinsam?</li><li>• Welches Interface-Pattern funktioniert für den gesamten Quadranten?</li><li>• Welche übergreifenden Design Principles lassen sich ableiten?</li></ul> |

## 5 Der Verifikationsquadrant im Enterprise SaaS Sales

Dieses Kapitel bildet das Herzstück der Arbeit. Es dokumentiert den iterativen Forschungsprozess und zeigt, wie sich der Verifikationsquadrant schrittweise mit Use Cases füllt – und wie daraus Erkenntnisse für das Interface-Design gewonnen werden.

### 5.1 Ausgangslage: Der leere Quadrant

Zu Beginn der Forschung steht der Quadrant leer. Wir wissen aus der Theorie (Wei), dass Aufgaben entlang zweier Achsen – Generierungsschwierigkeit und Verifikationsschwierigkeit – eingeordnet werden können. Aber welche konkreten Aufgaben im Enterprise SaaS Vertrieb wo einzuordnen sind, ist noch unklar.

Der leere Quadrant ist unser Ausgangspunkt und gleichzeitig unser Ziel: Am Ende der Arbeit soll er gefüllt sein – nicht nur mit Punkten, sondern mit Erkenntnissen, Design Patterns und einem tiefen Verständnis der jeweiligen Quadranten.

### 5.2 Erste Exploration: Interviews mit Enterprise-Vertriebspersonen

In der initialen Explorationsphase wurden **halbstrukturierte Interviews** mit Vertriebspersonen geführt, die Enterprise SaaS mit Average Contract Values im sechsstelligen Bereich verkaufen. Die Gespräche folgten einem narrativen Ansatz: Die Interviewpartner wurden gebeten, einen konkreten, aktuellen Deal von Anfang bis Ende durchzugehen und dabei besonders auf Herausforderungen, zeitintensive Aufgaben und frustrierende Aspekte einzugehen.

#### Zentrale Erkenntnis: Drei Töpfe

Aus den Interviews kristallisierten sich drei große Aufgabenbereiche heraus:

| Vertragliche Themen     | Mehrwert-Argumentation | Customer Success        |
|-------------------------|------------------------|-------------------------|
| • NDA Review & Approval | • ROI-Case erstellen   | • Support-Tickets lösen |

|                                                                                                                                                               |                                                                                                                                                                                       |                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Security Questionnaires</li> <li>• Contract Redlining</li> <li>• DPA-Prüfung</li> <li>• Compliance Checks</li> </ul> | <ul style="list-style-type: none"> <li>• Org-Struktur analysieren</li> <li>• SPIN/MEDDIC anwenden</li> <li>• Business Cases aufbauen</li> <li>• Entscheider identifizieren</li> </ul> | <ul style="list-style-type: none"> <li>• Bug-Reports bearbeiten</li> <li>• Feature Requests aufnehmen</li> <li>• Uptime-Kommunikation</li> <li>• NPS &amp; Zufriedenheit</li> </ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Diese drei Töpfe bilden das Grundgerüst für die weitere Einordnung in den Quadranten. Jeder Topf enthält Use Cases mit unterschiedlichen Verifikationseigenschaften – sie verteilen sich also über mehrere Quadranten.

### 5.3 Einordnung in den Quadranten

Auf Basis der Interviews und der eigenen Domänenenerfahrung wurden die identifizierten Use Cases in den Verifikationsquadranten eingeordnet. Diese erste Einordnung dient als Hypothese, die in den folgenden Iterationen validiert und ggf. angepasst wird.

#### Quadrant B: Schwer zu generieren, leicht zu verifizieren

Dies ist der **Sweet Spot für KI-Unterstützung**. Hier kann die KI die schwierige Generierungsarbeit übernehmen, während der Mensch als schneller, effizienter Verifikator agiert.

- **Security Questionnaires:** Die KI durchsucht interne Wissensdatenbanken, frühere Antworten und technische Dokumentation, um Antworten zu generieren. Ein Fachexperte kann die einzelnen Antworten relativ schnell auf Korrektheit prüfen – ist die Aussage technisch korrekt? Entspricht sie dem aktuellen Stand?
- **NDA Review:** Die KI analysiert eingehende NDAs, markiert potenziell kritische Klauseln und vergleicht mit Standardvorlagen. Ein juristisch geschulter Reviewer kann schnell prüfen: Sind die markierten Klauseln tatsächlich problematisch? Fehlt etwas?
- **Contract Redlining:** Die KI erstellt Redline-Vorschläge basierend auf unternehmensinternen Standards. Der Reviewer prüft die vorgeschlagenen Änderungen gegen bekannte Regeln.
- **Support-Ticket-Bearbeitung:** Die KI entwirft Lösungsvorschläge basierend auf ähnlichen vergangenen Tickets. Der Support-Mitarbeiter kann die vorgeschlagene Lösung schnell auf Plausibilität prüfen.

### Quadrant C: Leicht zu generieren, schwer zu verifizieren

Hier lauert die **größte Gefahr** für unkritischen KI-Einsatz. Behauptungen sind schnell aufgestellt, aber ihre Überprüfung erfordert Zeit, Daten oder ist prinzipiell unklar.

- **ROI-Cases:** Es ist leicht zu behaupten „Unsere Software spart Ihnen 30% der Kosten“. Aber die Verifikation erfordert Monate an Produktivdaten, Vergleichsmessungen und kontrollierte Bedingungen. Oft ist eine echte Verifikation erst nach der Kaufentscheidung möglich – wenn überhaupt.
- **Prognosen & Forecasts:** Eine Pipeline-Prognose ist schnell erstellt. Ob sie zutrifft, zeigt sich erst Monate später.
- **Langfristige Wertargumentationen:** Aussagen über strategischen Mehrwert, Marktpositionierung oder Effizienzgewinne sind leicht zu formulieren, aber notorisch schwer zu belegen.

### Quadrant D: Schwer zu generieren, schwer zu verifizieren

Der anspruchsvollste Bereich – hier sind sowohl die Lösungsfindung als auch die Überprüfung komplex:

- **Komplexe Pricing-Modelle:** Bei mehrdimensionalen Enterprise-Deals mit Custom-Pricing ist sowohl die Erstellung als auch die Bewertung der Angemessenheit schwierig.
- **Strategische Account-Planung:** Die optimale Strategie für einen Enterprise-Deal zu entwickeln erfordert tiefes Kontextwissen. Ob die Strategie optimal war, zeigt sich oft erst im Nachhinein.

### Quadrant A: Leicht zu generieren, leicht zu verifizieren

Use Cases, die gut automatisierbar sind und wenig menschliche Aufsicht erfordern:

- Standard-Status-Updates und Meeting-Zusammenfassungen
- Einfache Datenabfragen und CRM-Updates / Anreicherungen
- Template-basierte Kommunikation

## 5.4 Iterationen auf der Use-Case-Ebene

In den folgenden Unterkapiteln werden die wesentlichen Iterationsschleifen auf der Use-Case-Ebene dokumentiert. Jedes Unterkapitel folgt der Echelon-Struktur: Exploration, Hypothese, Prototyp, Evaluation, Learnings.



### 5.4.1 Security Questionnaires – Echelons

**Ausgangslage:** In 5 von 7 Interviews wurde das Beantworten von Security Questionnaires als einer der größten Zeitfresser genannt. Typischerweise müssen Vertriebler rare Ressourcen (Entwickler, Security-Experten) involvieren, was den Prozess verlangsamt und Frustration erzeugt.

#### Echelon 1 – Erste Hypothese:

- Hypothese: KI kann auf Basis vergangener Antworten und interner Dokumentation Antwortvorschläge generieren
- Prototyp: Einfaches Chat-Interface, das Fragen annimmt und Antworten vorschlägt
- Evaluation: Funktioniert grundsätzlich, aber das Interface ist nicht optimal – Reviewer verlieren den Überblick
- **Learning:** Der Review-Prozess braucht eine Struktur, die es ermöglicht, viele Antworten effizient nacheinander zu prüfen

#### Echelon 2 – Überarbeitetes Interface:

- Hypothese: Ein listenbasiertes Interface mit Sidebar ermöglicht effizienteres Reviewing
- Prototyp: Notification-Inbox-Stil mit Thread-Liste links, Detail-Ansicht rechts
- Evaluation: Deutlich bessere Effizienz, Reviewer können Use Cases schnell durcharbeiten
- **Learning:** Das Notification-Inbox-Pattern eignet sich hervorragend für Use Cases mit hohem Durchlauf und schneller Verifikation

*[Hinweis: Dieser Abschnitt wird in der finalen Thesis mit Screenshots, konkreten Nutzerfeedback-Zitaten und detaillierten Beschreibungen der vier Prototyp-Versionen ergänzt.]*

### 5.4.2 NDA Review & Approval – Echelons

**Ausgangslage:** NDAs vom Kunden müssen schnell geprüft und freigegeben werden. Verzögerungen hier blockieren den gesamten weiteren Prozess.

- KI kann kritische Terms identifizieren und mit Standards abgleichen
- Reviewer sieht markierte Stellen und kann schnell entscheiden
- **Parallele zum Security Questionnaire:** Ähnliches Verifikationsmuster – das Notification-Inbox-Pattern funktioniert auch hier

### 5.4.3 ROI-Case-Erstellung – Echelons

**Ausgangslage:** ROI-Cases sind fundamental anders: Die Generierung einer Behauptung ist einfach, aber die Verifikation ist extrem schwierig und zeitintensiv.

- Erste Versuche, ein ähnliches Review-Interface wie bei Security Questionnaires zu nutzen, scheitern
- Die Verifikation ist nicht binär (richtig/falsch), sondern graduell und kontextabhängig
- Erfordert Zugriff auf Produktdaten, Nutzungsstatistiken, Umfrageergebnisse
- **Learning:** Das Notification-Inbox-Pattern passt hier NICHT. Use Cases mit schwieriger Verifikation brauchen ein grundlegend anderes Interface-Paradigma – eines, das den Nutzer durch einen längeren Analyse- und Validierungsprozess begleitet.

*[Hinweis: Weitere Use-Case-Echelons (Customer Support Tickets, Org-Struktur-Analyse, etc.) werden in der finalen Thesis analog dokumentiert.]*

## 5.5 Iteration auf der Quadranten-Ebene

Nachdem mehrere Use Cases auf der Use-Case-Ebene untersucht wurden, werden die Erkenntnisse auf der Quadranten-Ebene verdichtet. Hier zeigen sich übergreifende Muster:

### **Erkenntnis 1 – Quadrant B (schwer generieren, leicht verifizieren):**

Alle untersuchten Use Cases in diesem Quadranten teilen eine gemeinsame Eigenschaft: Der Review-Prozess ist schnell, der Durchlauf ist hoch, und die KI-generierten Ergebnisse können effizient von einem Fachexperten geprüft werden. Das Notification-Inbox-Pattern hat sich konsistent als geeignet erwiesen.

### **Erkenntnis 2 – Quadrant C (leicht generieren, schwer verifizieren):**

Use Cases in diesem Quadranten erfordern ein fundamental anderes Interface-Paradigma. Die Verifikation ist nicht schnell und nicht binär. Stattdessen braucht es ein Interface, das den Nutzer durch einen strukturierten Analyse- und Validierungsprozess führt – möglicherweise über längere Zeiträume hinweg.

### **Erkenntnis 3 – Team Assignment als Querschnittsthema:**

Unabhängig vom Quadranten hat sich gezeigt, dass die korrekte Zuordnung von Aufgaben an die richtigen Personen (Team Assignment / Routing) ein kritischer Erfolgsfaktor ist. Die KI muss nicht nur Inhalte generieren, sondern auch wissen, wer der richtige Reviewer ist.

## 6 Design Patterns für Human-AI Interfaces

Auf Basis der iterativen Forschung lassen sich nun Design Patterns pro Quadrant ableiten. Diese Patterns sind keine starren Regeln, sondern empirisch fundierte Gestaltungsprinzipien, die als Ausgangspunkt für das Interface-Design dienen.

### 6.1 Pattern 1: Notification Inbox (Quadrant B)

**Anwendungsbereich:** Use Cases mit schwieriger Generierung, aber einfacher und schneller Verifikation.

**Beschreibung:**

Das Notification-Inbox-Pattern orientiert sich an E-Mail-Clients: Auf der linken Seite befindet sich eine Sidebar mit allen offenen Threads/Prozessen (z.B. „Security Questionnaire #4711 für Firma Acme“). Auf der rechten Seite wird das spezifische Interface für den jeweiligen Task-Typ angezeigt. Der Reviewer arbeitet sich durch die Inbox und kann schnell entscheiden: Approve, Reject, Modify.

**Charakteristiken:**

- Hoher Durchlauf: Viele einzelne Items können schnell abgearbeitet werden
- Binäre oder nahezu binäre Entscheidungen: Richtig/Falsch, Akzeptieren/Ablehnen
- KI hat Vorarbeit geleistet: Der Mensch verifiziert, nicht generiert
- Integriertes Team Assignment: Das System routet an den richtigen Experten
- Kontextanreicherung: Relevante Hintergrundinformationen werden mitgeliefert

**Geeignete Use Cases:**

- Security Questionnaire Review
- NDA Approval
- Contract Redline Review
- Support-Ticket-Prüfung
- Feature-Request-Triage

## 6.2 Pattern 2: Guided Analysis Workspace (Quadrant C)

**Anwendungsbereich:** Use Cases mit einfacher Generierung (Behauptungen), aber schwieriger, langwieriger Verifikation.

### **Beschreibung:**

Anders als das Notification-Inbox-Pattern ist dieses Pattern auf längere, tiefergehende Analyseprozesse ausgelegt. Es bietet dem Nutzer einen strukturierten Arbeitsbereich, in dem er über Zeit Evidenz sammeln, Daten aggregieren und Schritt für Schritt eine Verifikation aufbauen kann.

### **Charakteristiken:**

- Niedriger Durchlauf: Wenige, aber komplexe Items
- Graduelle Verifikation: Nicht binär, sondern ein Prozess des zunehmenden Vertrauens
- Datenüber-Zeit: Erfordert Zugriff auf historische und Echtzeitdaten
- Collaborative: Oft müssen mehrere Stakeholder beitragen
- Dashboard-ähnlich: Visualisierung von Trends, Metriken, Vergleichen

### **Geeignete Use Cases:**

- ROI-Case-Aufbau und -Validierung
- Langfristige Wertargumentation
- NPS-basierte Kundenzufriedenheits-Analyse
- Strategische Account-Planung

## 6.3 Pattern 3: Auto-Execute with Audit Trail (Quadrant A)

**Anwendungsbereich:** Use Cases mit einfacher Generierung UND einfacher Verifikation.

### **Beschreibung:**

Hier kann die KI nahezu autonom agieren. Das Interface zeigt dem Menschen lediglich an, was getan wurde (Audit Trail), und bietet die Möglichkeit zur Intervention bei Bedarf. Der Mensch ist kein aktiver Reviewer, sondern ein passiver Beobachter mit Veto-Recht.

## 6.4 Pattern 4: Structured Co-Creation (Quadrant D)

**Anwendungsbereich:** Use Cases, bei denen sowohl Generierung als auch Verifikation schwierig sind.

### Beschreibung:

Hier arbeiten Mensch und KI als gleichberechtigte Partner. Die KI unterstützt bei der Generierung, der Mensch bringt Kontextwissen und Urteilsfähigkeit ein. Verifikation ist ein iterativer, gemeinsamer Prozess. Das Interface muss intensive Interaktion und gemeinsames Arbeiten ermöglichen.

## 6.5 Zusammenfassung: Quadrant → Pattern Mapping

| Quadrant                | Design Pattern             | Rolle Mensch          | Rolle KI                        |
|-------------------------|----------------------------|-----------------------|---------------------------------|
| <b>A: Leicht/Leicht</b> | Auto-Execute + Audit Trail | Passiver Beobachter   | Autonomer Executor              |
| <b>B: Schwer/Leicht</b> | Notification Inbox         | Schneller Verifikator | Generierung + Routing           |
| <b>C: Leicht/Schwer</b> | Guided Analysis            | Aktiver Analyst       | Datensammlung + Strukturierung  |
| <b>D: Schwer/Schwer</b> | Structured Co-Creation     | Co-Creator            | Unterstützer + Sparring-Partner |

## 7 Diskussion & weiterführende Implikationen

### 7.1 Verschiebung des Tätigkeitsprofils

Eine zentrale Implikation der Ergebnisse betrifft die **Verschiebung des Tätigkeitsprofils** im Enterprise SaaS Vertrieb. Wenn KI die Aufgaben in Quadrant A und B zunehmend übernimmt oder zumindest die Generierungsarbeit leistet, dann verschieben sich die menschlichen Tätigkeiten unweigerlich in Richtung der Quadranten C und D.

Konkret bedeutet das: Wenn Security Questionnaires, NDA Reviews und Support-Tickets weitgehend von KI erledigt werden, wird der Mensch sich stärker auf jene Aufgaben konzentrieren müssen, die schwer zu verifizieren sind – ROI-Argumentation, strategische Planung, Beziehungsmanagement. Die Rolle verschiebt sich vom Ausführenden zum Strategen und Analysten.

### 7.2 Human-in-the-Loop vs. AI-in-the-Loop

Die Ergebnisse werfen eine provozierende Frage auf: Bis zu welchem Grad ist ein Human-in-the-Loop bei Use Cases mit einfacher Verifikation überhaupt sinnvoll?

Bei Security Questionnaires beispielsweise könnte perspektivisch die KI nicht nur die Antworten generieren, sondern auch die Verifikation übernehmen – etwa durch Kreuzvalidierung gegen mehrere unabhängige Quellen. Der Mensch wäre dann nicht mehr der Verifikator, sondern die KI würde eine Art AI-in-the-Loop für eine andere KI darstellen.

Gleichzeitig gibt es gute Gründe, den Menschen im Loop zu behalten: Verantwortlichkeit, Vertrauen des Kunden, regulatorische Anforderungen, und die Tatsache, dass KI-Systeme immer noch Halluzinationen produzieren können.

### 7.3 Agentic Business Process Management

Enterprise SaaS Sales ist ein hochgradig **kommunikationsintensiver Prozess**. E-Mails, Slack-Nachrichten, Meeting-Notizen – die Kommunikation zwischen Vertrieb, Kunde und internem Team bildet das Rückgrat des gesamten Sales-Prozesses. Hier knüpft das Konzept des **Agentic BPM** an.

Agentic BPM beschreibt die Idee, dass KI-Agenten eigenständig Geschäftsprozesse orchestrieren können – Aufgaben delegieren, Status tracken, Eskalationen auslösen. Die in dieser Arbeit identifizierten Design Patterns können als Bausteine für ein solches agentic BPM-System dienen:

- Das Notification-Inbox-Pattern für schnelle Review-Workflows
- Das Guided-Analysis-Pattern für komplexe Validierungsprozesse
- Team-Assignment-Logik für automatisiertes Routing

## 7.4 Guardrails & Vertrauen

Eine weitere Implikation betrifft die Gestaltung von Guardrails – also Leitplanken, die sicherstellen, dass KI-generierte Ergebnisse bestimmte Qualitätsstandards erfüllen. Die Verifikationsschwierigkeit eines Use Cases hat direkte Implikationen für die Art der benötigten Guardrails:

- **Quadrant A/B (leicht zu verifizieren):** Automatisierte Guardrails möglich (z.B. regelbasierte Checks, Unit-Test-ähnliche Validierung)
- **Quadrant C/D (schwer zu verifizieren):** Menschliche Guardrails erforderlich, ggf. ergänzt durch Confidence Scores und Unsicherheitsanzeigen

## 7.5 Wertschöpfungsketten und Kaskadeneffekte

Wenn bestimmte Aufgaben durch KI effizienter werden, hat das Kaskadeneffekte auf die gesamte Wertschöpfungskette im Vertrieb. Schnellere Security Questionnaires bedeuten schnellere Deal-Zyklen. Schnellere NDA-Approvals bedeuten früheren Zugang zu technischen Evaluierungen. Diese Beschleunigungen verändern nicht nur einzelne Aufgaben, sondern den gesamten Rhythmus und die Dynamik des Enterprise Sales-Prozesses.

Gleichzeitig können neue Bottlenecks entstehen: Wenn die „leichten“ Aufgaben schneller werden, werden die „schweren“ Aufgaben (ROI-Validierung, strategische Entscheidungen) zum neuen kritischen Pfad.



## 8 Fazit & Ausblick

Diese Arbeit hat gezeigt, dass Jason Weis Konzept der Verifikationsasymmetrie ein wertvolles analytisches Werkzeug für die Analyse und Gestaltung von Human-AI-Interfaces in komplexen Arbeitsdomänen ist. Am Beispiel des Enterprise SaaS Vertriebs wurde demonstriert, wie sich verschiedene Use Cases systematisch in einen Verifikationsquadranten einordnen lassen und wie daraus konkrete, quadrantenspezifische Design Patterns abgeleitet werden können.

### **Zentrale Ergebnisse:**

6. Der Verifikationsquadrant eignet sich als praktisches Framework zur Analyse und Strukturierung von Aufgaben im Enterprise SaaS Vertrieb
7. Vier Design Patterns wurden identifiziert: Notification Inbox (B), Guided Analysis (C), Auto-Execute (A) und Structured Co-Creation (D)
8. Die Position eines Use Cases im Quadranten hat direkte Implikationen für Interface-Design, Rolle des Menschen und Art der Guardrails
9. Der methodische Ansatz (DSR mit Echelons auf zwei Ebenen) hat sich als geeignet erwiesen, um sowohl tiefe Use-Case-Erkenntnisse als auch übergreifende Patterns zu gewinnen

### **Ausblick:**

Die Arbeit eröffnet mehrere Anschlussmöglichkeiten: Die Übertragung des Frameworks auf andere Domänen (z.B. Consulting, Rechtswesen, Healthcare), die empirische Validierung der Design Patterns in größerem Maßstab, und die Integration in agentic BPM-Systeme. Besonders spannend ist die Frage, wie sich der Quadrant selbst über Zeit verändert – wenn KI besser wird, verschieben sich Use Cases potenziell in Richtung leichter Verifikation, was wiederum neue Interface-Paradigmen ermöglicht.

## Anhang: Strategie für die Verteidigung

Für die Verteidigung der Thesis wird empfohlen, die Präsentation „**von hinten aufzuziehen**“ – also mit dem Ergebnis zu starten und dann den Weg dorthin zu erklären.

### Vorgeschlagener Aufbau der Präsentation (ca. 20-25 Min.)

10. **Eröffnung (2 Min.):** Den gefüllten Quadranten zeigen. „Das ist das Ergebnis meiner Arbeit. Lassen Sie mich erklären, wie ich hier hingekommen bin und was das bedeutet.“
11. **Das Problem (3 Min.):** Weis Verification Rule einführen. Sudoku-Beispiel. Warum ist das für die Praxis relevant?
12. **Die Domäne (3 Min.):** Enterprise SaaS Sales kurz erklären. Die drei Töpfe zeigen. Warum ist dieser Bereich ein ideales Anwendungsfeld?
13. **Die Methodik (3 Min.):** DSR mit Echelons erklären. Zwei-Ebenen-Ansatz visualisieren.
14. **Der Weg zum Ergebnis (5-7 Min.):** Ausgewählte Echelons zeigen. Den Quadranten schrittweise aufbauen. Kontrast zwischen Security Questionnaire (leicht zu verifizieren) und ROI-Case (schwer zu verifizieren).
15. **Die Design Patterns (3-4 Min.):** Die vier Patterns vorstellen. Mockup-Screenshots zeigen. Gegenüberstellung Notification Inbox vs. Guided Analysis.
16. **Diskussion & Implikationen (3-4 Min.):** Task Shifting, Human-in-the-Loop vs. AI-in-the-Loop, Agentic BPM. Der Bogen zum großen Ganzen.
17. **Abschluss (1 Min.):** Zurück zum gefüllten Quadranten. Was haben wir gelernt? Was kommt als Nächstes?

**Kern-Narrative der Verteidigung:** „Die Schwierigkeit der Verifikation bestimmt, wie das Interface zwischen Mensch und KI gestaltet sein muss. Ich habe das am Beispiel Enterprise SaaS Sales untersucht, iterativ Design Patterns entwickelt und zeige, welche fundamentalen Implikationen sich daraus für die Zukunft wissensintensiver Arbeit ergeben.“